

Sesión 11/12 — 10 ejercicios progresivos POO básica en C++

IS6041 · Clases, objetos,
encapsulamiento, setters, getters y
constructores

Ejercicio 1 — Universidad: perfil de estudiante

- **Contexto**
- La oficina de tutoría de una universidad quiere registrar el nombre, la edad y el promedio de un estudiante de primer ciclo.
- **Consigna**
- Crea una clase Estudiante con atributos privados nombre, edad y promedio. Implementa setters y getters. En main, crea un objeto, asigna valores válidos y muestra la información.
- **Enfócate en:**
 - Clase y objeto
 - Atributos privados
 - Setters y getters básicos

Ejercicio 2 — Tienda deportiva: control de stock

- **Contexto**
- Una tienda deportiva en línea vende zapatillas urbanas a jóvenes de 18 a 20 años y quiere evitar registrar stock negativo.
- **Consigna**
- Crea una clase Producto con nombre, precio y stock. El setter del stock no debe aceptar valores negativos. Muestra un mensaje si el valor es inválido. Luego prueba la clase en main.
- **Enfócate en:**
 - Encapsulamiento
 - Validación con setters
 - Reglas de negocio simples

Ejercicio 3 — Streaming: plan de suscripción

- **Contexto**
- Una app de streaming para universitarios ofrece planes Básico, Estándar y Premium.
- **Consigna**
- Crea una clase Suscripcion con usuario, plan y pagoMensual. Agrega un método mostrarResumen() que imprima un resumen bonito. Usa constructor con parámetros para inicializar el objeto.
- **Enfócate en:**
 - Constructor con parámetros
 - Métodos públicos
 - Uso de objetos en contexto real

Ejercicio 4 — Videojuego: jugador y puntaje

- **Contexto**
- Un videojuego universitario necesita registrar el nickname de un jugador, su nivel y su puntaje acumulado.
- **Consigna**
- Crea una clase Jugador con constructor, getters y un método subirNivel() que aumente el nivel en 1. Además, crea un método sumarPuntos(int puntos). Muestra el estado antes y después de actualizar.
- **Enfócate en:**
 - Métodos que modifican el estado
 - Constructores
 - Getters

Ejercicio 5 — Wallet universitaria

- **Contexto**
- Una billetera digital para estudiantes permite recargar saldo y pagar consumos dentro del campus.
- **Consigna**
- Crea una clase Wallet con saldo privado. Agrega métodos recargar(double monto) y pagar(double monto). El pago solo debe realizarse si hay saldo suficiente. Muestra mensajes apropiados.
- **Enfócate en:**
 - Encapsulamiento con reglas de negocio
 - Métodos con decisiones internas
 - Validación de saldo

Ejercicio 6 — Rappi universitario: pedido express

- **Contexto**
- Un estudiante pide comida por delivery desde una app. El pedido tiene cliente, total, estado y costo de envío.
- **Consigna**
- Crea una clase Pedido con atributos privados. Implementa métodos calcularTotalFinal() y cambiarEstado(string nuevoEstado). Solo permite estados: Pendiente, En camino, Entregado.
- **Enfócate en:**
 - Métodos con validación de texto
 - Estados controlados
 - Comportamiento derivado

Ejercicio 7 — Gimnasio joven: membresía

- **Contexto**
- Un gimnasio universitario ofrece membresías por meses. Cada alumno tiene nombre, meses pagados y si su membresía está activa.
- **Consigna**
- Crea una clase Membresia con constructor, getters y un método renovar(int meses). Si los meses a renovar no son positivos, no debe aceptarlos. También crea un método desactivar().
- **Enfócate en:**
 - Cambios de estado controlados
 - Constructores
 - Métodos coherentes con negocio

Ejercicio 8 — Tienda gamer: carrito de compra

- **Contexto**
- Una tienda gamer para jóvenes vende accesorios. Cada carrito debe registrar el nombre del producto, cantidad y precio unitario.
- **Consigna**
- Crea una clase ItemCarrito con constructor, getters y método calcularSubtotal(). Luego, en main, crea dos objetos y muestra el subtotal de cada uno.
- **Enfócate en:**
 - Objetos múltiples
 - Métodos de cálculo
 - Uso repetido de una misma clase

Ejercicio 9 — Universidad: curso con vacantes

- **Contexto**
- Una universidad necesita controlar las vacantes de un curso electivo popular entre estudiantes de 18 a 20 años.
- **Consigna**
- Crea una clase Curso con nombre, vacantes y matriculados. Implementa un método matricular() que solo incremente matriculados si todavía hay vacantes. Agrega también mostrarEstado().
- **Enfócate en:**
 - Reglas internas de coherencia
 - Métodos de negocio
 - Protección del estado

Ejercicio 10 — Proyecto integrador: perfil de creador de contenido

- **Contexto**

- Un joven creador de contenido administra su perfil digital con nombre del canal, seguidores, ingresos mensuales y estado de monetización.

- **Consigna**

- Crea una clase CreadorContenido con atributos privados y constructor. Implementa métodos sumarSeguidores(int), registrarIngreso(double), activarMonetizacion() y mostrarPerfil(). La monetización solo puede activarse si tiene al menos 1000 seguidores.

- **Enfócate en:**

- Integración de toda la sesión
- Reglas complejas
- Diseño más completo de clase